



ΔΙΑΧΕΙΡΙΣΗ ΣΥΝΘΕΤΩΝ ΔΕΔΟΜΕΝΩΝ

Εργασία 2

ΔΗΜΗΤΡΑ ΧΡΙΣΤΙΝΑ ΓΚΑΡΑΒΕΛΑ
ΑΜ:5051

Περιεχόμενα

Μέρος 1.....	2
Λίγα Λόγια για την άσκηση	2
Υλοποίηση του R-tree	2
Μέρος 2.....	11
Λίγα λόγια για την άσκηση.....	11
Ανάλυση κώδικα	11
Μέρος 3.....	16
Λίγα Λόγια για την Άσκηση	16
Ανάλυση κώδικα	16

Μέρος 1

Λίγα Λόγια για την άσκηση

Για το πρώτο σκέλος της εργασίας υλοποιήθηκε R-Tree πάνω σε ένα σύνολο από πολύγωνα που περιγράφονται από τα αρχεία *coords.txt* (όλες οι κορυφές) και *offsets.txt* (δείκτες αρχής/τέλους κάθε πολυγώνου). Αρχικά, για κάθε αντικείμενο υπολογίζεται το ελάχιστο ορθογώνιο (Minimum Bounding Rectangles ή MBR), στη συνέχεια το κέντρο αυτού του MBR μετασχηματίζεται σε κωδικό Morton (Z-order) μέσω της έτοιμης ρουτίνας `interleave_latlng`. Τα αντικείμενα ταξινομούνται σύμφωνα με τον Morton-κώδικα ώστε χωρικά γειτονικά MBRs να αποθηκευτούν διαδοχικά και να τοποθετηθούν στο ίδιο φύλλο του δέντρου. Ακολουθεί «πακετάρισμα» σε κόμβους χωρητικότητας $MAX = 20$ με ελάχιστο $MIN = 8$ εγγραφές. Η διαδικασία εξελίσσεται ιεραρχικά, με κάθε νέο επίπεδο να δημιουργείται από τα MBRs του προηγούμενου, έως ότου προκύψει μία και μοναδική ρίζα. Τελικό αποτέλεσμα: στον τερματικό εκτυπώνεται ο αριθμός κόμβων ανά επίπεδο, ενώ στο αρχείο *Rtree.txt* παράγεται μία γραμμή ανά κόμβο με τη μορφή `[isnonleaf, node-id, [[id, [xlow, xhigh, ylow, yhigh]], ...]]`.

Υλοποίηση του R-tree

Ο κώδικας υλοποιεί ένα πλήρες R-Tree και οργανώνεται γύρω από οκτώ συνάρτησεις που αλληλοσυμπληρώνονται.

morton()

Μετατρέπει το κέντρο κάθε MBR σε κωδικό Morton (Z-order). Καλεί τη συνάρτηση `interleave_latlng` της βιβλιοθήκης *pymorton* με σειρά (y, x), η οποία επιστρέφει μια αλυσίδα ψηφίων 0-3. Κάθε ψηφίο είναι ένας «τετραδικός» (base-4) κώδικας που κωδικοποιεί δύο bits του Morton-code (00, 01, 10, 11). Η `rjust()` είναι μια μέθοδος η οποία ελέγχει το τρέχον μήκος του `raw`, αν είναι μικρότερο από το 32, προσθέτει αριστερά όσα επιπλέον μηδενικά χρειάζονται. Επιστρέφει το νέο string χωρίς να μεταβάλει το αρχικό. Αν το αρχικό string έχει ήδη μήκος ≥ 32 , επιστρέφεται όπως είναι. Ο σκοπός είναι όλοι οι κωδικοί να έχουν σταθερό μήκος 32 χαρακτήρων, για να μπορούμε να κάνουμε απλή λεξικογραφική ταξινόμηση.

```
8
9  def morton(lat: float, lon: float) -> str:
10      raw = interleave_latlng(lat, lon)
11      return raw.rjust(32, "0")
12
```

mbf()

Η συνάρτηση `mbf(points)` δέχεται ως είσοδο μια λίστα από κορυφές (σημεία) ενός πολυγώνου, όπου κάθε σημείο αναπαρίσταται ως ζεύγος (x, y) συντεταγμένων. Στόχος της συνάρτησης είναι να υπολογίσει και να επιστρέψει το ελάχιστο περιβάλλον ορθογώνιο (MBR) που περιλαμβάνει όλα αυτά τα σημεία. Για να το πετύχει αυτό, χωρίζει τις x-συντεταγμένες και τις y-συντεταγμένες των σημείων σε δύο ξεχωριστές λίστες. Στη συνέχεια, χρησιμοποιεί τις συναρτήσεις `min` και `max` για να βρει το μικρότερο και το μεγαλύτερο x (δηλαδή τα αριστερότερο και δεξιότερο άκρα του MBR), το μικρότερο και το μεγαλύτερο y (δηλαδή τα κατώτατο και ανώτατο όρια του MBR). Αυτά τα τέσσερα άκρα (`xmin`, `xmax`, `ymin`, `ymax`) συνθέτουν το MBR, το οποίο επιστρέφεται ως τετράδα.

```
18 def mbr(points: List[Tuple[float, float]]) -> MBR:
19     xs = []
20     ys = []
21
22     for (x, y) in points:
23         xs.append(x)
24         ys.append(y)
25
26     xmin = min(xs)      # πιο αριστερό x
27     xmax = max(xs)      # πιο δεξί x
28     ymin = min(ys)      # πιο χαμηλό y
29     ymax = max(ys)      # πιο ψηλό y
30
31     return (xmin, xmax, ymin, ymax)
```

Η συνάρτηση `pack()` σπάει μια λίστα σε «μπλοκ» χωρητικότητας $MAX = 20$ και εγγυάται το κατώφλι $MIN = 8(0,4 \cdot MAX)$. Συγκεκριμένα, ξεκινά διασπώντας τη λίστα σε τμήματα μεγέθους MAX (20). Η μεταβλητή `i` ξεκινά από 0. Αυτή η μεταβλητή δείχνει από ποια θέση της λίστας θα ξεκινήσουμε κάθε νέο «κόψιμο», όσο το `i` είναι μικρότερο από το μήκος της λίστας, συνεχίζουμε να κόβουμε υπολίστες. Παίρνουμε μια υπολίστα που ξεκινά από το στοιχείο στη θέση `i` και περιλαμβάνει μέχρι `i + MAX` στοιχεία, δηλαδή παίρνουμε το στοιχείο στη θέση `i`, `i+1`, ..., μέχρι `i+MAX-1` ή μέχρι το τέλος της λίστας αν έχει λιγότερα. Αφού κόψαμε ένα κομμάτι, προχωράμε τη μεταβλητή `i` κατά MAX θέσεις, για να ετοιμαστούμε να κόψουμε το επόμενο κομμάτι.

Στη συνέχεια ελέγχουμε αν υπάρχουν τουλάχιστον δύο ομάδες (groups), γιατί αν έχουμε μόνο μία ομάδα, δεν έχει νόημα να «δανειστεί» από προηγούμενη αφού δεν υπάρχει. Παίρνουμε την τελευταία ομάδα (αυτή που μπορεί να τις λείπουν στοιχεία) που φτιάξαμε και την προτελευταία ομάδα (αυτή που μπορεί να δώσει στοιχεία). Ελέγχουμε αν το τελευταίο group έχει λιγότερα από το κατώφλι (MIN) στοιχεία. Αν ναι, πρέπει να ενισχυθεί για να είναι αποδεκτό. Υπολογίζουμε πόσα στοιχεία του λείπουν για να φτάσει το ελάχιστο MIN ($need = MIN - len(last_group)$) και παίρνουμε τα τελευταία `need` στοιχεία από τον

προηγούμενο group (previous_group) για να τα δώσουμε (to_move = previous_group[-need:]). Προσθέτουμε τα στοιχεία που πήραμε στην αρχή του last_group (To [:0] = ... σημαίνει “κάνε insert μπροστά” το οποίο γίνεται για να διατηρήσουμε τη συνολική σειρά ταξινόμησης) και αφαιρούμε αυτά τα στοιχεία από τον previous_group, γιατί τα δώσαμε ήδη (groups[-2] = previous_group[: -need]).

```
34 def pack(lst):
35     groups = []
36     i = 0
37     while i < len(lst):
38         groups.append( lst[i : i + MAX] ) # κόψε υπολίστα
39         i += MAX                          # προχώρησε κατά MAX
40
41     if len(groups) > 1:
42         last_group = groups[-1]
43         previous_group = groups[-2]
44
45         if len(last_group) < MIN:
46             need = MIN - len(last_group)
47             to_move = previous_group[-need:]
48             last_group[:0] = to_move
49             groups[-2] = previous_group[: -need]
50
51     return groups
52
```

mbr_union(): Παίρνει μια λίστα entries, όπου κάθε στοιχείο είναι: (child_id, mbr) και το mbr είναι τετράδα: (xmin, xmax, ymin, ymax). Σκοπός είναι να βρεθεί το κοινό MBR που καλύπτει όλα τα παιδιά, αυτό θα αποθηκευτεί στον γονικό κόμβο.

Αρχικά αποθηκεύουμε το MBR του πρώτου παιδιού και στη συνέχεια, μέσω βρόγχου for το προσαρμόζουμε συγκρίνοντάς το με τα MBR των υπόλοιπων παιδιών. Σε κάθε βήμα ενημερώνουμε τα όρια ώστε να καλύπτονται όλα τα παιδιά μέχρι εκείνη τη στιγμή. Η ίδια διαδικασία επαναλαμβάνεται και για κάθε ανώτερο επίπεδο του δέντρου, καθώς κάθε νέος γονικός κόμβος απαιτεί τον υπολογισμό ενός MBR που να περικλείει όλα τα παιδιά του, άρα γίνεται εκ νέου χρήση της συνάρτησης mbr_union.

```
54 def mbr_union(entries):
55     first_mbr = entries[0][1]
56     xmin, xmax, ymin, ymax = first_mbr
57     for _cid, mbr in entries[1:]:
58         child_xmin, child_xmax, child_ymin, child_ymax = mbr
59         xmin = min(xmin, child_xmin)
60         xmax = max(xmax, child_xmax)
61         ymin = min(ymin, child_ymin)
62         ymax = max(ymax, child_ymax)
63     return (xmin, xmax, ymin, ymax)
64
```

Η **load()** παραλαμβάνει δύο αρχεία της εκφώνησης και επιστρέφει λίστα τριάδων (z, object_id, mbr) ταξινομημένη ως προς z_code.

Δημιουργείται μια άδεια λίστα `points`, όπου θα αποθηκευτούν όλα τα σημεία από το `coords.txt` ως (x,y) ζεύγη (`Tuple[float,float]`). Ανοίγεται το αρχείο `coords.txt` για διάβασμα, με χειρισμό μέσω του file handler `fh`. Για κάθε γραμμή `line` μέσα στο αρχείο `coords.txt`:

- καθαρίζονται τα κενά/νέες γραμμές (**`strip()`**) και γίνεται **`split()`** της γραμμής στα δύο (x και y) με διαχωριστή το κόμμα «,». Το **`x_str`** (το πρώτο κομμάτι (πριν το κόμμα)) και το **`y_str`** (το δεύτερο κομμάτι (μετά το κόμμα)) μετατρέπονται σε float (η **`split()`** επιστρέφει strings) και προστίθενται ως (x, y) tuple στη λίστα `points`.

```
65 def load(coords_file: str, offsets_file: str):
66     points: List[Tuple[float, float]] = []
67
68     with open(coords_file, "r") as fh:
69         for line in fh:
70             x_str, y_str = line.strip().split(",")
71             points.append( (float(x_str), float(y_str)) )
```

Δημιουργείται κενή λίστα `objects`, που θα αποθηκεύσει για κάθε αντικείμενο ένα (z, `object_id`, `mbr`) και ανοίγεται το δεύτερο αρχείο `offsets.txt` για διάβασμα, όπου ορίζονται τα πολύγωνα μέσω δεικτών. Για κάθε γραμμή του αρχείου ακολουθείται η παρακάτω διαδικασία:

- Αρχικά, με την `line.strip()` αφαιρούνται τυχόν κενά, tabs ή αλλαγές γραμμής στην αρχή και το τέλος της γραμμής. Αν η γραμμή είναι κενή, την αγνοούμε και προχωράμε στην επόμενη (**`continue`**). Διαχωρίζουμε τη γραμμή σε τρία κομμάτια με `line.split(",")` : **`obj_id_str`**, **`to_start_str`** και το **`end_str`**, που είναι σε μορφή **string**. Στη συνέχεια, τα strings αυτά μετατρέπονται σε ακέραιους (`int`) ώστε να μπορούν να χρησιμοποιηθούν ως αριθμοί: **`obj_id`** (το αναγνωριστικό του αντικειμένου (πολύγωνου)), **`start`** (η θέση στη λίστα `points` όπου ξεκινάει το πολύγωνο) και **`end`**(η θέση όπου τελειώνει το πολύγωνο). Έπειτα, με το `points[start : end + 1]`, ανακτώνται όλα τα σημεία που ανήκουν στο συγκεκριμένο πολύγωνο. Το `end + 1` είναι απαραίτητο για να πάρουμε και το στοιχείο στη θέση `end`. Από τα σημεία του πολυγώνου υπολογίζεται το **MBR** χρησιμοποιώντας τη βοηθητική συνάρτηση **`mbr()`**, η οποία βρίσκει τα ελάχιστα και μέγιστα x και y των σημείων. Κατόπιν, υπολογίζεται το **κέντρο του MBR** (`cx`, `cy`): $cx = (x_{min} + x_{max}) / 2$ και $cy = (y_{min} + y_{max}) / 2$. Το κέντρο αυτό χρησιμοποιείται για τον υπολογισμό του Morton κωδικού (Z-order curve), μέσω της συνάρτησης `morton(cy, cx)`. Το αποτέλεσμα, δηλαδή η τριάδα «z, `object_id`, `mbr`», αποθηκεύεται στη λίστα `objects`.

```

81     with open(offsets_file, "r") as fh:
82         for line in fh:
83             line = line.strip()
84             if line == "":
85                 continue
86
87             obj_id_str, start_str, end_str = line.split(",")
88
89             obj_id = int(obj_id_str)
90             start = int(start_str)
91             end = int(end_str)
92
93             poly_pts = points[start : end + 1]      # σημεία πολυγώνου
94             mb = mbr(poly_pts)                    # MBR του πολυγώνου
95
96             cx = (mb[0] + mb[1]) / 2
97             cy = (mb[2] + mb[3]) / 2
98             z = morton(cy, cx)
99
100            objects.append((z, obj_id, mb))

```

Αφού ολοκληρωθεί η επεξεργασία όλων των γραμμών, η λίστα `objects` ταξινομείται με βάση τον Morton κωδικό με την χρήση της `itemgetter(0)`, η οποία λέει να παίρνεις κάθε φορά το 0-στο στοιχείο από κάθε αντικείμενο της λίστας (`z`). Έτσι, τα πολυγωνικά αντικείμενα να τοποθετούνται διαδοχικά στον χώρο σύμφωνα με τη Z-order. Τέλος, η ταξινομημένη λίστα `objects` επιστρέφεται για να χρησιμοποιηθεί στο χτίσιμο του R-tree.

```

102     objects.sort(key=itemgetter(0))    # ταξινόμηση βάσει z
103     return objects

```

Η συνάρτηση **build()** υλοποιεί την κατασκευή του R-tree με στρατηγική bottom-up bulk-loading και παίρνει σαν όρισμα την λίστα `objects` από την **load()**. Ξεκινά δημιουργώντας τα φύλλα, ομαδοποιώντας τα ήδη ταξινομημένα αντικείμενα με τη βοήθεια της `pack()`. Στη συνέχεια, επαναλαμβάνει τη διαδικασία δημιουργίας επιπέδων εφαρμόζοντας `pack` και `mbr_union` διαδοχικά, έως ότου απομείνει μόνο ένας κόμβος που λειτουργεί ως ρίζα.

Αρχικά, δημιουργείται μια κενή λίστα `levels`, η οποία θα περιέχει όλα τα επίπεδα του R-tree, όπου κάθε επίπεδο είναι μια λίστα από κόμβους. Παράλληλα, αρχικοποιείται και η λίστα `leaves`, που θα συγκεντρώσει όλους τους κόμβους-φύλλα του δέντρου (δηλαδή το κατώτερο επίπεδο). Στη συνέχεια, καλείται η `pack()` για να χωρίσει τα αντικείμενα `objects` σε ομάδες μεγέθους έως και `MAX = 20`. Για κάθε τέτοια ομάδα, επεξεργαζόμαστε κάθε αντικείμενο (που είναι τριάδα (`z, oid, mbr`)), κρατάμε το `oid` και το `mbr` και τα αποθηκεύουμε σε μια νέα λίστα `entries` ως (`object_id, mbr`). Με βάση αυτά τα `entries`, δημιουργείται ένα λεξικό (`dictionary`) που περιγράφει έναν κόμβο-φύλλο του R-tree, το οποίο προστίθεται στην λίστα `leaves`. Συγκεκριμένα, «leaf»: True δηλώνει ότι αυτός ο κόμβος είναι φύλλο (δηλαδή περιέχει κανονικά αντικείμενα, όχι άλλους κόμβους) και «entries»: `entries` αποθηκεύει τις εγγραφές που ανήκουν στο φύλλο, κάθε εγγραφή είναι ένα (`object_id, mbr`) (δηλαδή ID του αντικειμένου και το MBR του).

Τέλος, το επίπεδο των φύλλων αποθηκεύεται στη θέση `levels[0]`, αποτελώντας το πρώτο επίπεδο του R-tree.

```
105 def build(objects):
106     levels = []
107     # φύλλα
108     leaves = []
109     for group in pack(objects):          # κόψε τα objects 20-20
110         entries = []
111         for _z, oid, mb in group:
112             entries.append( (oid, mb) )
113         leaves.append( {"leaf": True, "entries": entries} )
114     levels = [leaves]
```

Ξεκινάμε τη διαδικασία δημιουργίας των εσωτερικών επιπέδων του R-tree από το `level_index = 1`. Όσο το τελευταίο επίπεδο (`levels[-1]`) περιέχει περισσότερους από έναν κόμβους, σημαίνει ότι απαιτείται η κατασκευή ανώτερων επιπέδων. Ορίζουμε το `prev_level` ως το τρέχον επίπεδο (είτε τα φύλλα είτε ένα ενδιάμεσο επίπεδο γονικών κόμβων) και δημιουργούμε μια κενή λίστα `parents` όπου θα αποθηκεύσουμε τους νέους γονικούς κόμβους.

Για κάθε ομάδα κόμβων στο `prev_level`, την οποία πακετάρουμε σε ομάδες μεγέθους έως `MAX=20` με τη βοήθεια της `pack()`, επεξεργαζόμαστε κάθε παιδί της ομάδας:

- για κάθε παιδί-κόμβο υπολογίζουμε το συνολικό MBR που καλύπτει όλα τα `entries` του μέσω της `mbr_unio(child[«entries»])` και δημιουργούμε ένα προσωρινό entry (`None`, `συνολικό_mbr`), όπου το `None` θα αντικατασταθεί αργότερα με το πραγματικό `child_id`.

Έπειτα, δημιουργούμε έναν νέο γονικό κόμβο σε μορφή λεξικού, με `«leaf»: False` για να δηλώσουμε ότι δεν είναι φύλλο, `«entries»: parent_entries` για να καταγράψουμε τα συνολικά MBRs των παιδιών του, και `«children»: group` για να διατηρήσουμε προσωρινά τους δείκτες στα παιδιά-κόμβους. Ο κάθε νέος γονικός προστίθεται στη λίστα `parents` και το `level_index` αυξάνεται κατά 1.

Η διαδικασία επαναλαμβάνεται αναδρομικά μέχρι το τελευταίο επίπεδο να περιέχει έναν μόνο κόμβο, τη ρίζα του δέντρου. Τελικά, επιστρέφουμε τη δομή `levels`, όπου `levels[0]` περιέχει τα φύλλα, `levels[1]`, `levels[2]`, κλπ. τα ενδιάμεσα επίπεδα γονικών κόμβων και `levels[-1][0]` αποτελεί τη μοναδική ρίζα του R-tree.


```

116     # γονικά
117     level_index = 1
118     while len(levels[-1]) > 1:          # μέχρι να μείνει μία ρίζα
119         prev_level = levels[-1]        # κόμβοι τρέχοντος επιπέδου
120         parents = []
121
122         for group in pack(prev_level):
123             parent_entries = []
124             for child in group:
125                 parent_entries.append( (None, mbr_union(child["entries"])) )
126             parents.append({
127                 "leaf": False,
128                 "entries": parent_entries,
129                 "children": group
130             })
131         levels.append(parents)
132         level_index += 1
133
134     return levels

```

Η **flatten()** μετατρέπει τη δομή `levels`, μια λίστα επιπέδων όπου κάθε επίπεδο περιέχει λίστες από λεξικά-κόμβους, σε έναν ενιαίο flat πίνακα `nodes`, όπου η θέση κάθε κόμβου στη λίστα λειτουργεί και ως το `node-id` του. Η μετατροπή αυτή είναι απαραίτητη ώστε να παραχθεί το τελικό R-tree σε μορφή συμβατή με την έξοδο που απαιτείται (`Rtree.txt`), όπου κάθε κόμβος αναφέρεται μέσω ακέραιου `node-id` και όχι μέσω αναφορών σε αντικείμενα.

Αρχικά, δημιουργείται μια κενή λίστα `nodes`, στην οποία προστίθενται διαδοχικά όλοι οι κόμβοι από όλα τα επίπεδα του `levels`, διατηρώντας τη σειρά ώστε να ευθυγραμμίζεται με τα μελλοντικά `node-ids`. Στη συνέχεια, αρχικοποιούμε ένα κενό λεξικό `idmap`. Για κάθε κόμβο `node` στη λίστα `nodes`, παίρνουμε και το `i`, δηλαδή τη θέση του κόμβου μέσα στη λίστα. Η `id()` είναι ενσωματωμένη συνάρτηση της Python που επιστρέφει το μοναδικό αναγνωριστικό ενός αντικειμένου κατά τη διάρκεια ζωής του. Είναι ένας ακέραιος αριθμός που αντιπροσωπεύει τη θέση του αντικειμένου στη μνήμη. Καταχωρούμε στον χάρτη ότι το `id` του αντικειμένου `node` αντιστοιχεί στη θέση `i` μέσα στη λίστα `nodes`. Δηλαδή κρατάμε ποιο είναι το `node_id` κάθε κόμβου.

```

137     def flatten(levels):
138         nodes = []
139         for lvl in levels:
140             for node in lvl:
141                 nodes.append(node)
142         idmap = {}
143         for i, node in enumerate(nodes):
144             idmap[id(node)] = i
145

```

Για κάθε κόμβο του πίνακα:

- Αν είναι φύλλο (`leaf=True`), τα `entries` του παραμένουν αμετάβλητα, καθώς ήδη αναφέρονται σε αντικείμενα (`object_ids`).

- Αν είναι εσωτερικός κόμβος (leaf=False), παίρνουμε το mbr_child, δηλαδή το MBR που καλύπτει το αντίστοιχο παιδί. Παράλληλα, μέσω του node["children"], παίρνουμε τον ίδιο τον child κόμβο (child_node). Χρησιμοποιώντας τον χάρτη idmap, βρίσκουμε το αριθμητικό αναγνωριστικό (child_id) που αντιστοιχεί σε αυτόν τον κόμβο. Δημιουργούμε ένα νέο entry της μορφής (child_id, mbr_child) και το προσθέτουμε στη λίστα new_entries.
- Μετά το πέρας της επεξεργασίας, αντικαθιστούμε τα παλιά entries του κόμβου με τη νέα λίστα new_entries, η οποία πλέον περιέχει μόνο έγκυρα child IDs αντί για προσωρινά None. Το προσωρινό πεδίο children διαγράφεται, καθώς πλέον κάθε child αναφέρεται έμμεσα μέσω του child_id.

Τέλος, επιστρέφεται η ενημερωμένη λίστα nodes, η οποία είναι πλέον μονοδιάστατη (flat), με όλα τα entries σωστά ενημερωμένα με child IDs και έτοιμη για αποθήκευση στο αρχείο Rtree.txt.

```

146     for node in nodes:
147         if node["leaf"]:
148             continue
149
150         new_entries = []
151         for i in range(len(node["entries"])):
152             mbr_child = node["entries"][i][1]          # Πάρε το mbr από το entry
153             child_node = node["children"][i]           # Πάρε το αντίστοιχο παιδί
154             child_id = idmap[id(child_node)]           # Βρες το id
155             new_entries.append((child_id, mbr_child))  # Φτιάξε το νέο entry
156
157         node["entries"] = new_entries
158         del node["children"]
159
160     return nodes

```

Η συνάρτηση **save()** διατρέχει τη λίστα nodes και γράφει στο αρχείο Rtree.txt μία γραμμή ανά κόμβο, ακολουθώντας ακριβώς το ζητούμενο format [isnonleaf, node-id, [[id, [xlow, xhigh, ylow, yhigh]], ...]]. Πιο αναλυτικά, η save ανοίγει το αρχείο path σε λειτουργία εγγραφής ("w") με κωδικοποίηση UTF-8, αν το αρχείο υπάρχει ήδη, το περιεχόμενό του αντικαθίσταται. Στη συνέχεια, διατρέχει όλους τους κόμβους της λίστας nodes, παίρνοντας ταυτόχρονα το node_id (δηλαδή τη θέση του κόμβου στη λίστα) και το περιεχόμενό του node.

Για κάθε κόμβο:

- Αν είναι φύλλο (leaf=True), το isnonleaf τίθεται σε 0, αλλιώς (αν είναι εσωτερικός κόμβος), το isnonleaf γίνεται 1.
- Για κάθε entry του κόμβου (που είναι ένα (id, mbr) ζεύγος) μετατρέπει το MBR (που είναι tuple) σε λίστα [xmin, xmax, ymin, ymax] για να έχουμε σωστή μορφή στο αρχείο μας.

- Δημιουργεί μία συμβολοσειρά line που περιλαμβάνει το isnonleaf, το node_id και τη λίστα των entries nice_entries, όλα τοποθετημένα μέσα σε αγκύλες.
- Γράφει τη γραμμή στο ανοιχτό αρχείο out_file.

Αφού διαχειριστεί όλους τους κόμβους, η συνάρτηση κλείνει το αρχείο.

```

163 def save(nodes, path="Rtree.txt"):
164     out_file = open(path, "w", encoding="utf-8")
165     for node_id, node in enumerate(nodes):
166
167         # αν είναι φύλλο -> isnonleaf = 0
168         # αν είναι εσωτερικός -> isnonleaf = 1
169         if node["leaf"]:
170             isnonleaf = 0
171         else:
172             isnonleaf = 1
173
174         nice_entries = []
175         for entry_id, mbr in node["entries"]:
176             nice_entries.append([entry_id, list(mbr)])
177
178         line = f"[{isnonleaf}, {node_id}, {nice_entries}]\n"
179         out_file.write(line)
180
181     out_file.close()

```

Η συνάρτηση **main()** δέχεται τα δύο αρχεία εισόδου, εκτελεί διαδοχικά τα βήματα load→build → save και εκτυπώνει στον τερματικό τον αριθμό κόμβων που δημιουργήθηκαν σε κάθε επίπεδο του δέντρου.

```

183 def main():
184     coords = sys.argv[1]
185     offsets = sys.argv[2]
186
187     objects = load(coords, offsets)
188     levels = build(objects)
189
190
191     print()
192     for lvl, nodes in enumerate(levels):
193         n = len(nodes)
194         word = "node" if n == 1 else "nodes"
195         print(f"{n} {word} at level {lvl}")
196
197     save(flatten(levels))
198
199 if __name__ == "__main__":
200     main()

```

Μέρος 2

Λίγα λόγια για την άσκηση

Το δεύτερο σκέλος της εργασίας αφορά αποτίμηση ερωτήσεων εύρους (**range queries**) πάνω στο R-Tree που δημιουργήθηκε στο Μέρος 1. Για καθεμία από τις γραμμές του Rqueries.txt (μορφή *xlow ylow xhigh yhigh*) πρέπει να βρούμε όλα τα πολυγωνικά αντικείμενα των οποίων το MBR τέμνει το ορθογώνιο W και να εκτυπώσουμε γραμμή (πλήθος): ids.

Ανάλυση κώδικα

Η συνάρτηση **intersects(a,b)** ελέγχει αν δύο ορθογώνια a και b τέμνονται, δηλαδή αν έχουν έστω και ένα κοινό σημείο στο επίπεδο. Δύο MBRs τέμνονται όταν οι προβολές τους επικαλύπτονται και στους δύο άξονες. Αποθηκεύουμε τα όρια των δυο ορθογώνιων a και b, ax1, ax2: το χαμηλό και το υψηλό όριο στο x του πρώτου ορθογωνίου, ay1, ay2: το χαμηλό και το υψηλό όριο στο y του πρώτου ορθογωνίου και αντίστοιχα για το b. Ελέγχουμε αν οι δύο προβολές (στον x και y άξονα) επικαλύπτονται:

Για τον άξονα x:

- $ax1 \leq bx2$: το αριστερό άκρο του a είναι πριν ή πάνω στο δεξί άκρο του b.
- $ax2 \geq bx1$: το δεξί άκρο του a είναι μετά ή πάνω στο αριστερό άκρο του b.

Για τον άξονα y:

- $ay1 \leq by2$: το κάτω άκρο του a είναι πριν ή πάνω στο πάνω άκρο του b.
- $ay2 \geq by1$: το πάνω άκρο του a είναι μετά ή πάνω στο κάτω άκρο του b.

Αν ισχύουν και οι δύο έλεγχοι (x και y άξονας), τότε τα ορθογώνια τέμνονται και επιστρέφουμε True. Αν έστω ένας από τους δύο ελέγχους αποτύχει, επιστρέφουμε False, άρα τα ορθογώνια δεν τέμνονται.

```
13 def intersects(a: MBR, b: MBR) -> bool:
14     ax1, ax2, ay1, ay2 = a
15     bx1, bx2, by1, by2 = b
16     x_overlap = (ax1 <= bx2) and (ax2 >= bx1)
17     y_overlap = (ay1 <= by2) and (ay2 >= by1)
18     return x_overlap and y_overlap
```

range_search()

Αποτελεί τον αναδρομικό πυρήνα του range-query. Αν ο τρέχων κόμβος είναι φύλλο, προσθέτει όλα τα αντικείμενα που τέμνουν το παράθυρο στα αποτελέσματα. Αν είναι εσωτερικός, καλεί αναδρομικά μόνο τα παιδιά που επικαλύπτονται με το παράθυρο. Συγκεκριμένα, παίρνει σαν παράμετρους: **tree** (όλο το R-Tree ως λίστα κόμβων), **root_id** (το id του κόμβου από τον οποίο ξεκινάμε (συνήθως η ρίζα)), **window** (το ορθογώνιο ερώτησης (W) που ψάχνουμε να βρούμε ποια MBR το τέμνουν) και **out** (μια λίστα όπου θα αποθηκευτούν τα object ids που ταιριάζουν). Αρχικά, βρίσκουμε τον τρέχοντα κόμβο με βάση το root_id.

- Αν ο κόμβος είναι φύλλο (leaf=True), τότε περιέχει αντικείμενα (object_ids και MBRs), όχι άλλους κόμβους. Διατρέχουμε όλα τα αντικείμενα (entries) του κόμβου (oid: το id του αντικειμένου, mbr: το MBR του αντικειμένου), αν το MBR του αντικειμένου τέμνει το window (με χρήση της intersects), τότε προσθέτουμε το oid στα αποτελέσματα out.
- Διαφορετικά, διατρέχουμε τα παιδιά του (child_id: το id του παιδιού, mbr: το MBR που καλύπτει το παιδί κόμβο). Αν το MBR του παιδιού τέμνει το window (με χρήση της intersects, τότε αναδρομικά καλούμε range_search πάνω στο παιδί αυτό. Δηλαδή, κατεβαίνουμε μόνο σε παιδιά που μπορεί να περιέχουν αντικείμενα μέσα στο window, όχι σε όλα.

```
25 def range_search(tree: List[Node], root_id: int,  
26                 window: MBR, out: List[int]) -> None:  
27     node = tree[root_id]  
28  
29     if node['leaf']:  
30         # κάθε entry = [object_id, MBR]  
31         for oid, mbr in node['entries']:  
32             if intersects(mbr, window):  
33                 out.append(oid)  
34     else:  
35         # κάθε entry = [child_id, MBR]  
36         for child_id, mbr in node['entries']:  
37             if intersects(mbr, window):  
38                 range_search(tree, child_id, window, out)  
39
```

Η συνάρτηση **load_tree()** μετατρέπει το αρχείο Rtree.txt από απλό κείμενο σε μια δομή Python, όπου κάθε κόμβος είναι αποθηκευμένος σε λίστα.

Ορίζουμε τη load_tree που παίρνει το path προς το αρχείο Rtree.txt και επιστρέφει μια λίστα κόμβων (List[Node]). Κάθε γραμμή του αρχείου είναι ήδη στη μορφή [isnonleaf, node-id, entries]. Αρχικοποιούμε μια κενή λίστα nodes, όπου θα τοποθετήσουμε κάθε κόμβο. Ανοίγουμε το αρχείο path για ανάγνωση με κωδικοποίηση UTF-8 και διαβάζουμε

γραμμή-γραμμή το αρχείο. Με την χρήση της `line.strip()` αφαιρούμε κενά ή νέα γραμμή (`\n`) από την αρχή και το τέλος της γραμμής `line`. Η `literal_eval` (από το `ast module`) παίρνει μία συμβολοσειρά που μοιάζει με Python λίστα και τη μετατρέπει σε πραγματική λίστα Python. Η έξοδος του `literal_eval` είναι μια λίστα με 3 στοιχεία:

- **isnonleaf**: ακέραιος 0 ή 1 (0 = φύλλο, 1 = εσωτερικός κόμβος),
- **node_id**: ακέραιος, που είναι το ID του κόμβου στο δέντρο και
- **entries**: λίστα εγγραφών:
 - Αν είναι φύλλο: [(object_id, [xmin,xmax,ymin,ymax]), ...]
 - Αν είναι εσωτερικός κόμβος: [(child_id, [xmin,xmax,ymin,ymax]), ...]

Βεβαιωνόμαστε ότι η λίστα `nodes` είναι αρκετά μεγάλη για να μπορούμε να αποθηκεύσουμε τον τρέχοντα κόμβο, αν όχι, γεμίζουμε με `None` μέχρι να φτάσουμε το απαραίτητο μήκος. Δημιουργούμε ένα λεξικό που αντιπροσωπεύει τον κόμβο. Το πεδίο «leaf» παίρνει την τιμή `True` αν είναι φύλλο (`isnonleaf == 0`), αλλιώς `false`. Το πεδίο «entries» περιέχει τη λίστα εγγραφών, όπως διαβάστηκε από το αρχείο (ήδη σωστά μορφοποιημένη).

Τελικά επιστρέφουμε τη λίστα `nodes` με όλους τους κόμβους του R-Tree φορτωμένους στη RAM.

```
37 def load_tree(path: str) -> List[Node]:
38     nodes: List[Node] = []
39
40     with open(path, encoding='utf-8') as f:
41         for line in f:
42             parts = literal_eval(line.strip())
43             isnonleaf = parts[0]
44             node_id = parts[1]
45             entries = parts[2]
46
47             while len(nodes) <= node_id:
48                 nodes.append(None)
49             nodes[node_id] = {
50                 'leaf': (isnonleaf == 0),
51                 'entries': entries
52             }
53
54     return nodes
```

Η **`run_queries()`** ορίζει μια συνάρτηση που εκτελεί όλες τις ερωτήσεις εύρους από ένα αρχείο, με ορίσματα `tree`: μια λίστα από κόμβους R-tree (όπως φορτώθηκε από `load_tree()`) και `queries_path`: το μονοπάτι του αρχείου με τις ερωτήσεις. Συγκεκριμένα, ανοίγει το αρχείο των ερωτήσεων χρησιμοποιώντας `with` ώστε να κλείσει αυτόματα το αρχείο όταν τελειώσει η χρήση του. Διαβάζει γραμμή-γραμμή, αποθηκεύοντας ταυτόχρονα τον αριθμό γραμμής (`line_no`) και το περιεχόμενο της γραμμής (`line`) μέσω της `enumerate()`.

Αν μια γραμμή είναι κενή (μόνο κενά ή νέα γραμμή), αγνοείται με `continue`. Διαχωρίζει τη γραμμή σε 4 τιμές (`x_min`, `y_min`, `x_max`, `y_max`) χωρισμένες με κενό (`line.split()`), και τις μετατρέπει από `str` σε `float` μέσω της χρήσης της `map()`. Φτιάχνουμε το ορθογώνιο που θα χρησιμοποιηθεί για αναζήτηση στο R-tree (`window`) ως `tuple`.

Στη συνέχεια, δημιουργείται μια κενή λίστα `results`, που θα περιέχει τα object IDs των αντικειμένων που τέμνουν το τρέχον παράθυρο. Καλούμε την `range_search()` αναδρομικά, ξεκινώντας από τη ρίζα του R-tree (η τελευταία θέση της λίστας `tree = len(tree) - 1`) και αποθηκεύουμε τα αποτελέσματα (τα object IDs που τέμνουν το παράθυρο) στη λίστα `results`. Δεδομένου ότι στο Μέρος 1 οι κόμβοι γράφονται σειριακά (φύλλα πρώτα, ρίζα τελευταία), η ρίζα βρίσκεται απευθείας στη θέση `root_id = len(tree) - 1`, χωρίς ανάγκη αναζήτησης.

Μετατρέπουμε τα IDs από αριθμούς σε strings και τα ενώνουμε με κόμμα ανάμεσα. Τέλος, εκτυπώνουμε την απάντηση της ερώτησης που περιλαμβάνει:

- τον αριθμό της γραμμής ερώτησης (`line_no`),
- το πλήθος των αποτελεσμάτων σε παρένθεση (`len(results)`),
- και τη λίστα των IDs.

```
62 def run_queries(tree: List[Node], queries_path: str) -> None:
63     with open(queries_path) as f:
64         for line_no, line in enumerate(f):
65             if not line.strip():
66                 continue
67
68             x_low, y_low, x_high, y_high = map(float, line.split())
69             window = (x_low, x_high, y_low, y_high)
70
71             results: List[int] = []
72             range_search(tree, root_id=len(tree) - 1, # ρίζα = τελευταία γραμμή
73                         window=window, out=results)
74
75             ids_txt = ",".join(map(str, results))
76             print(f"{line_no} {len(results)}: {ids_txt}")
77
```

main()

Συνδέει όλα τα στάδια: φορτώνει το δέντρο, εκτελεί όλες τις ερωτήσεις και τυπώνει τα αποτελέσματα. Συγκεκριμένα, με την χρήση της `sys.argv` παίρνουμε τα ορίσματα από την γραμμή εντολών, με `sys.argv[1]` το αρχείο `RTree.txt` και `sys.argv[2]` το `Rqueries.txt`.

Καλείται η συνάρτηση `load_tree()` για να φορτωθεί το R-tree από το αρχείο, και το αποτέλεσμα αποθηκεύεται στη μεταβλητή `tree` (λίστα κόμβων). Εκτελούνται όλες οι ερωτήσεις εύρους που περιέχει το αρχείο `queries_file`, με βάση το δέντρο `tree`. Για κάθε ερώτηση, εντοπίζονται τα αντικείμενα που τέμνουν το ερωτηματικό παράθυρο και εκτυπώνονται τα αποτελέσματα (`run_queries`).

```
79 def main():
80     rtree_file = sys.argv[1]
81     queries_file = sys.argv[2]
82     tree = load_tree(rtree_file)
83     run_queries(tree, queries_file)
84
85
86 if __name__ == "__main__":
87     main()
88
```


Μέρος 3

Λίγα Λόγια για την Άσκηση

Στο τρίτο μέρος της εργασίας, ζητείται η υλοποίηση αλγορίθμου best-first search για k-Nearest Neighbor (k-NN) queries πάνω σε R-tree. Σκοπός είναι να εντοπιστούν τα k πλησιέστερα αντικείμενα (object IDs) σε ένα σημείο αναφοράς $q=(x,y)$, βάσει των ελαχίστων αποστάσεων από τα MBRs των πολυγώνων. Ο αλγόριθμος πρέπει να υλοποιείται με βάση την κλασική προσέγγιση best-first. Για τη διαχείριση των υποψήφιων MBRs αξιοποιείται ουρά προτεραιότητας ταξινομημένη ως προς την ελάχιστη ευκλείδεια απόσταση από το σημείο q . Η ουρά περιέχει τόσο εσωτερικούς κόμβους (node MBRs) όσο και φύλλα (object MBRs). Κάθε φορά που εξάγεται από την ουρά ένα object MBR, αυτό προστίθεται στα αποτελέσματα. Η διαδικασία επαναλαμβάνεται μέχρι να βρεθούν τα k πλησιέστερα αντικείμενα.

Ανάλυση κώδικα

Ορίζεται η συνάρτηση **mindist()** που παίρνει ως είσοδο ένα σημείο pt (π.χ. (x, y)) και ένα MBR m (τετράδα $xmin, xmax, ymin, ymax$). Επιστρέφει έναν αριθμό float, δηλαδή την ευκλείδεια απόσταση από το σημείο στο MBR. Συγκεκριμένα, αναθέτουμε τις συντεταγμένες του σημείου στις μεταβλητές x και y , και τα όρια του ορθογώνιου MBR στα $xmin, xmax, ymin, ymax$. Αρχικά, υπολογίζουμε την οριζόντια απόσταση (dx) από το ορθογώνιο:

- Αν το x βρίσκεται μέσα στο εύρος του MBR στον x -άξονα, τότε η απόσταση ισούται με μηδέν ($dx=0$).
- Αν το x είναι στα αριστερά του MBR ($x < xmin$), τότε η απόσταση ισούται με $dx = xmin - x$.
- Το x είναι στα δεξιά του MBR ($x > xmax$), τότε η απόσταση ισούται με $dx = x - xmax$

Έπειτα, υπολογίζουμε την κάθετη απόσταση (dy) από το ορθογώνιο:

- Αν το y βρίσκεται μέσα στο εύρος του MBR στον y -άξονα, τότε η απόσταση ισούται με μηδέν ($dy=0$).
- Αν το y είναι κάτω από το MBR ($y < ymin$), τότε η απόσταση ισούται με $dy = ymin - y$.
- Αν το y είναι πάνω από το MBR ($y > ymax$), τότε η απόσταση ισούται με $dy = y - ymax$.

Τέλος, επιστρέφουμε την ευκλείδεια απόσταση από το πλησιέστερο σημείου του MBR.

```

12 def mindist(pt: Tuple[float, float], m: MBR) -> float:
13     x, y = pt
14     xmin, xmax, ymin, ymax = m
15     if xmin <= x <= xmax:
16         dx = 0
17     elif x < xmin:
18         dx = xmin - x
19     else:
20         dx = x - xmax
21     if ymin <= y <= ymax:
22         dy = 0
23     elif y < ymin:
24         dy = ymin - y
25     else:
26         dy = y - ymax
27
28     return math.sqrt(dx**2 + dy**2)
29

```

Η **load_rtree()** είναι μια συνάρτηση που φορτώνει ένα R-tree από αρχείο κειμένου. Το αρχείο αυτό είναι στη μορφή που παρήγαγε το `meros1.py`. Συγκεκριμένα, αρχικοποιούμε ένα άδαιο λεξικό `nodes` που θα αντιστοιχεί τα `node_id` με λεξικό που περιέχει αν είναι φύλλο («leaf»: True/False) και τις εγγραφές του («entries»). Ορίζουμε ένα `root_id` που θα κρατήσει το id της ρίζας του R-tree (τελευταίος κόμβος στο αρχείο) και ανοίγουμε το αρχείο. Για κάθε γραμμή του αρχείου, μετατρέπουμε την γραμμή (line) από string που μοιάζει με Python λίστα σε κανονική λίστα Python με τη `ast.literal_eval()` και αποθηκεύουμε στα πεδία `isnonleaf`: 0 σημαίνει φύλλο, 1 σημαίνει εσωτερικός, `node_id`: αριθμός του κόμβου και `entries`: λίστα εγγραφών (είτε προς αντικείμενα είτε προς παιδιά). Στη συνέχεια, φτιάχνουμε ένα λεξικό για τον κόμβο με πεδία «leaf»: True αν είναι φύλλο (δηλ. αν `isnonleaf == 0`) και «entries»: η λίστα εγγραφών αυτού του κόμβου. Αυτό το λεξικό το τοποθετούμε στη θέση `node_id` του `nodes`. Τέλος, ενημερώνουμε το `root_id` με το τρέχον `node_id` και επιστρέφει:

- Το λεξικό `nodes` με όλους τους κόμβους
- Το `root_id`, δηλαδή το `node_id` της ρίζας

```

38 def load_rtree(path: str):
39     nodes: Dict[int, Dict] = {}
40     root_id = None
41     with open(path) as f:
42         for line in f:
43             isnonleaf, node_id, entries = ast.literal_eval(line)
44             nodes[node_id] = {"leaf": isnonleaf == 0, "entries": entries}
45             root_id = node_id
46     return nodes, root_id
47

```

Ορίζουμε τη συνάρτηση **knn_search()** η οποία υλοποιεί best-first k-NN αναζήτηση σε R-tree χρησιμοποιώντας ουρά προτεραιότητας.

Δέχεται παραμέτρους: nodes: το R-tree σε μορφή λεξικού (node_id → κόμβος), root_id: το id της ρίζας (συνήθως ο τελευταίος κόμβος), q: το σημείο αναφοράς (x, y) ως πλειάδα (tuple), k: πόσους κοντινότερους γείτονες θέλουμε να βρούμε και επιστρέφει λίστα από k object_ids

Αρχικοποιούμε ουρά προτεραιότητας (min-heap). Κάθε στοιχείο είναι τετράδα:

- dist: απόσταση του σημείου q από το MBR
- is_leaf_entry: αν είναι φύλλο (True) ή εσωτερικός κόμβος (False)
- node_or_obj_id: είτε το object_id (αν φύλλο) είτε το node_id (αν εσωτερικός)
- mbr: το ίδιο το MBR

Παίρνουμε τον κόμβο της ρίζας από το λεξικό nodes. Για κάθε entry της ρίζας (που είναι είτε object είτε child) υπολογίζουμε την mindist(q, m), η οποία είναι η ελάχιστη ευκλείδεια απόσταση σημείου q από το MBR και προσθέτουμε την εγγραφή στην ουρά. Αρχικοποιείται η λίστα αποτελεσμάτων, όπου θα αποθηκεύσουμε τα k πλησιέστερα αντικείμενα (results). Όσο η ουρά δεν είναι άδεια και δεν έχουμε βρει ακόμα k αντικείμενα βγάζουμε από την ουρά την εγγραφή με τη μικρότερη απόσταση (best-first) και αναλύουμε τα πεδία της εγγραφής. Αν είναι αντικείμενο (δηλαδή φύλλο), τότε έχουμε βρει έναν γείτονα και τον αποθηκεύουμε και συνεχίζουμε στην επόμενη επανάληψη. Αν δεν είναι φύλλο (εσωτερικός κόμβος), τότε επεκτείνουμε το cid και παίρνουμε τον child κόμβο. Για κάθε εγγραφή του κόμβου αυτού υπολογίζουμε απόσταση από το q, και τη βάζουμε στην ουρά, για να εξεταστεί μελλοντικά.

Μόλις έχουμε βρει k κοντινότερα αντικείμενα, τα επιστρέφουμε.

```
49 def knn_search(nodes, root_id: int, q: Tuple[float, float], k: int) -> List[int]:
50     heap = []
51     root = nodes[root_id]
52     for cid, m in root["entries"]:
53         heapq.heappush(heap, (mindist(q, m), root["leaf"], cid, m))
54
55     result = []
56     while heap and len(result) < k:
57         dist, is_leaf_entry, cid, m = heapq.heappop(heap)
58
59         if is_leaf_entry:
60             result.append(cid)
61             continue
62
63         child = nodes[cid]
64         for nid, cm in child["entries"]:
65             heapq.heappush(heap, (mindist(q, cm), child["leaf"], nid, cm))
66
67     return result
```

Ορίζουμε τη βασική συνάρτηση **main()**, που είναι το σημείο εκκίνησης του προγράμματος.

Από τη λίστα `sys.argv` παίρνουμε το πρώτο όρισμα, που είναι το όνομα του αρχείου R-tree, το δεύτερο όρισμα είναι το αρχείο με τις ερωτήσεις κοντινών γειτόνων και το τρίτο όρισμα είναι ο αριθμός `k`, δηλαδή πόσους πλησιέστερους γείτονες θέλουμε, το οποίο μετατρέπουμε σε `int`. Καλούμε τη συνάρτηση `load_rtree()` για να διαβάσει το R-tree από αρχείο και να μας επιστρέψει ένα `dictionary` με όλους τους κόμβους του δέντρου (`nodes`) και το ID της ρίζας του δέντρου (`root_id`). Ανοίγουμε το αρχείο ερωτήσεων και το διατρέχουμε γραμμή γραμμή κρατώντας και τον αριθμό της γραμμής (`enumerate`). Καθαρίζουμε κάθε γραμμή από τυχόν περιττά κενά ή νέα γραμμή (`\n`) και ελέγχουμε αν είναι κενή, τότε συνεχίζουμε στην επόμενη. Χωρίζουμε τη γραμμή στα δύο αριθμητικά μέρη (`x`, `y` συντεταγμένες σημείου) και δημιουργούμε το σημείο ερώτησης `q`, αφού τα έχουμε μετατρέψει από `strings` σε `float`. Καλούμε τη βασική συνάρτηση `knn_search()` για να πάρουμε τα `k` πλησιέστερα `object-ids` στο σημείο `q` και εκτυπώνουμε τα IDs των αποτελεσμάτων στη μορφή «0: 123,456,789» όπου 0 είναι η γραμμή ερώτησης, και τα υπόλοιπα είναι τα `k` πιο κοντινά `object-ids`.

```
70 def main():
71     rtree_file = sys.argv[1]
72     queries_file = sys.argv[2]
73     k = int(sys.argv[3])
74
75     nodes, root_id = load_rtree(rtree_file)
76
77
78     with open(queries_file) as f:
79         for idx, line in enumerate(f):
80             line = line.strip()
81             if not line:
82                 continue
83             x_str, y_str = line.split()
84             q = (float(x_str), float(y_str))
85             ids = knn_search(nodes, root_id, q, k)
86             print(f"{idx}: {' '.join(map(str, ids))}")
87
88 if __name__ == "__main__":
89     main()
```